

Agenda

- Omówienie kolokwium
- Testy

Testy



Testy

- Testy jednostkowe
- Testy integracyjne
- Testy funkcjonalne
- Testy akceptacyjne
- Testy wydajnościowe
- Testy regresji
- Testy bezpieczeństwa
- ...

JUnit

```
<dependency>  
  <groupId>org.junit.jupiter</groupId>  
  <artifactId>junit-jupiter-api</artifactId>  
  <version>5.10.2</version>  
  <scope>test</scope>  
</dependency>
```

5 JUnit 5

[JUnit 4](#)

The 5th major version of the programmer-friendly testing framework for Java and the JVM

[User Guide](#)

[Javadoc](#)

[Code & Issues](#)

[Q & A](#)

[Support JUnit](#)

JUnit team's statement on the war in Ukraine

As human beings, we stand with Ukraine and condemn the Russian government's war against the Ukrainian people, including our own colleagues and their families.

[Donate to UN's Ukraine Humanitarian Fund](#)

Latest Release

Jupiter [v5.11.0-M1](#) Vintage [v5.11.0-M1](#)
Platform [v1.11.0-M1](#)

JUnit artifacts are deployed to Maven Central and can be downloaded using the above links. All files are

Tablica indeksowana miesiącami

```
public class MonthlyArray<T> {  
    T data[];  
  
    public MonthlyArray(LocalDate begin, LocalDate end) {  
        int years = end.getYear() - begin.getYear();  
        int months = end.getMonthValue() - begin.getMonthValue();  
        months += years * 12;  
        data = (T[]) new Object[months];  
    }  
  
    public int size() {  
        return data.length;  
    }  
}
```

indeks	rok.miesiąc	obiekt
0	2024.3	null
1	2024.4	null
2	2024.5	null
3	2024.6	null

Test rozmiaru

```
import org.junit.jupiter.api.Test;

public class MonthlyArrayTest {
    @Test
    void arraySize() {
        MonthlyArray<Object> monthlyArray = new MonthlyArray<>(
            LocalDate.of(2024, 3, 1),
            LocalDate.of(2024, 7, 1)
        );

        int size = monthlyArray.size();
        assertEquals(4, size);
    }
}
```

indeks	rok.miesiąc	obiekt
0	2024.3	null
1	2024.4	null
2	2024.5	null
3	2024.6	null

Uruchomienie testu

```
$ mvn test
```

```
[INFO] -----
```

```
[INFO]  T E S T S
```

```
[INFO] -----
```

```
[INFO] Running MonthlyArrayTest
```

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0 -- in MonthlyArrayTest
```

```
[INFO]
```

```
[INFO] Results:
```

```
[INFO]
```

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

Błąd w programie

```
public class MonthlyArray<T> {  
    T data[];  
  
    public MonthlyArray(LocalDate begin, LocalDate end) {  
        int years = end.getYear() - begin.getYear();  
        int months = end.getMonthValue() - begin.getMonthValue();  
        months = years * 12;  
        data = (T[]) new Object[months];  
    }  
  
    public int size() {  
        return data.length;  
    }  
}
```

indeks	rok.miesiąc	obiekt
0	2024.3	null
1	2024.4	null
2	2024.5	null
3	2024.6	null

Log z niezaliczonego testu

```
$ mvn test
```

```
Tests run: 1, Failures: 1, Errors: 0, Skipped: 0 <<< FAILURE! -- in MonthlyArrayTest
```

```
MonthlyArrayTest.arraySize -- Time elapsed: 0.016 s <<< FAILURE!
```

```
org.opentest4j.AssertionFailedError: expected: <4> but was: <0>
```

```
    at org.junit.jupiter.api.AssertionFailureBuilder.build(AssertionFailureBuilder.java:151)
```

```
    at org.junit.jupiter.api.AssertionFailureBuilder.buildAndThrow  
        (AssertionFailureBuilder.java:132)
```

```
    at org.junit.jupiter.api.AssertEquals.failNotEqual(AssertEquals.java:197)
```

```
    at org.junit.jupiter.api.AssertEquals.assertEquals(AssertEquals.java:150)
```

```
    at org.junit.jupiter.api.AssertEquals.assertEquals(AssertEquals.java:145)
```

```
    at org.junit.jupiter.api.Assertions.assertEquals(Assertions.java:531)
```

```
    at MonthlyArrayTest.arraySize(MonthlyArrayTest.java:21)
```

```
    at java.base/java.lang.reflect.Method.invoke(Method.java:580)
```

```
    at java.base/java.util.ArrayList.forEach(ArrayList.java:1597)
```

Indeks tablicy

```
public class MonthlyArray<T> {  
    int beginYear, beginMonth;  
    T data[];  
  
    public MonthlyArray(LocalDate begin, LocalDate end) {  
        beginYear = begin.getYear();  
        beginMonth = begin.getMonthValue();  
  
        int years = end.getYear() - begin.getYear();  
        int months = end.getMonthValue() - begin.getMonthValue();  
        months += years * 12;  
        data = (T[]) new Object[months];  
    }  
  
    public int index(int year, int month) {  
        return (year - beginYear) * 12 + month - beginMonth;  
    }  
}
```

Test indeksu

```
public class MonthlyArrayTest
{
    @Test
    void index() {
        MonthlyArray<Object> monthlyArray = new MonthlyArray<>(
            LocalDate.of(2020, 1, 1),
            LocalDate.of(2024, 7, 1)
        );
        int index = monthlyArray.index(2024, 1);
        assertEquals(48, index);
    }
}
```

Indeks spoza przedziału

```
public class MonthlyArray<T> {  
  
    public int index(int year, int month) throws IndexOutOfBoundsException {  
        int index = (year - beginYear) * 12 + month - beginMonth;  
        if (index < 0 || index >= size())  
            throw new IndexOutOfBoundsException();  
        return index;  
    }  
}
```

```
$ mvn test
```

```
Tests run: 1, Failures: 0, Errors: 1, Skipped: 0 <<< FAILURE! -- in MonthlyArrayTest.index -  
- <<< ERROR!  
java.lang.IndexOutOfBoundsException  
    at MonthlyArray.index(MonthlyArray.java:26)  
    ...
```

Test rzucenia wyjątku

```
@Test
void indexNegative() {
    MonthlyArray<Object> monthlyArray = new MonthlyArray<>(
        LocalDate.of(2024, 1, 1),
        LocalDate.of(2024, 3, 1)
    );
    assertThrows(IndexOutOfBoundsException.class, () -> monthlyArray.index(2023, 12));
}
```

```
@Test
void indexTooHigh() {
    MonthlyArray<Object> monthlyArray = new MonthlyArray<>(
        LocalDate.of(2024, 1, 1),
        LocalDate.of(2024, 3, 1)
    );
    assertThrows(IndexOutOfBoundsException.class, () -> monthlyArray.index(2024, 3));
}
```

Test z wieloma asercjami

```
@Test
void multipleIndices() {
    MonthlyArray<Object> monthlyArray = new MonthlyArray<>(
        LocalDate.of(2010, 1, 1),
        LocalDate.of(2024, 1, 1)
    );

    assertEquals(35, monthlyArray.index(2012, 12));
    assertEquals(146, monthlyArray.index(2022, 3));
    assertEquals(124, monthlyArray.index(2020, 5));
}
```

Parametryzacja testów - zależności

```
<dependency>  
  <groupId>org.junit.jupiter</groupId>  
  <artifactId>junit-jupiter-api</artifactId>  
  <version>5.10.2</version>  
  <scope>test</scope>  
</dependency>  
  
<dependency>  
  <groupId>org.junit.jupiter</groupId>  
  <artifactId>junit-jupiter-params</artifactId>  
  <version>5.10.2</version>  
  <scope>test</scope>  
</dependency>
```

Kolejność dat

```
public MonthlyArray(LocalDate begin, LocalDate end) {  
    if(begin.isAfter(end))  
        throw new IllegalArgumentException();  
  
    beginYear = begin.getYear();  
    beginMonth = begin.getMonthValue();  
  
    int years = end.getYear() - begin.getYear();  
    int months = end.getMonthValue() - begin.getMonthValue();  
    months += years * 12;  
    data = (T[]) new Object[months];  
}
```


Test sparametryzowany napisami

```
@ParameterizedTest
@ValueSource(strings = {"2019-01-01", "2014-12-01", "2023-01-31"})
void illegalDateOrder (String endDateString) {
    LocalDate beginDate = LocalDate.of(2023,2,1);
    LocalDate endDate = LocalDate.parse(endDateString);

    assertThrows(
        IllegalArgumentException.class,
        () -> new MonthlyArray<>(beginDate, endDate)
    );
}
```

Test nierzucania wyjątku

```
@ParameterizedTest
@ValueSource(strings = {"2024-01-01", "2026-12-01", "2123-01-31"})
void correctDateOrder (String endDateString) {
    LocalDate beginDate = LocalDate.of(2023,2,1);
    LocalDate endDate = LocalDate.parse(endDateString);

    assertDoesNotThrow(
        () -> new MonthlyArray<>(beginDate, endDate)
    );
}
```

Test sparmetryzowany metoda

```
private static Stream<LocalDate> exampleDates() {  
    return Stream.of(  
        LocalDate.of(2024, 12, 1),  
        LocalDate.of(2026, 3, 1),  
        LocalDate.of(2123, 5, 1)  
    );  
}
```

```
@ParameterizedTest  
@MethodSource ("exampleDates")  
void correctDateOrder (LocalDate endDate) {  
    LocalDate beginDate = LocalDate.of(2023,2,1);  
    assertDoesNotThrow(() -> new MonthlyArray<>(beginDate, endDate));  
}
```

Użycie tablicy do parametryzacji testu

```
private static Stream<LocalDate[]> exampleDateArrays() {  
    return Stream.of(  
        new LocalDate[]{LocalDate.of(2024, 12, 1), LocalDate.of(2024, 12, 1)},  
        new LocalDate[]{LocalDate.of(1990, 6, 1), LocalDate.of(2000, 5, 1)},  
        new LocalDate[]{LocalDate.of(2100, 12, 1), LocalDate.of(2200, 12, 1)}  
    );  
}  
  
@ParameterizedTest  
@MethodSource("exampleDateArrays")  
void correctDateOrder(LocalDate beginDate, LocalDate endDate) {  
    assertDoesNotThrow(() -> new MonthlyArray<>(beginDate, endDate));  
}
```

Typ Arguments

```
private static Stream<Arguments> exampleDateArguments() {  
    return Stream.of(  
        arguments(LocalDate.of(2024, 12, 1), LocalDate.of(2024, 12, 1)),  
        arguments(LocalDate.of(1990, 6, 1), LocalDate.of(2000, 5, 1)),  
        arguments(LocalDate.of(2100, 12, 1), LocalDate.of(2200, 12, 1))  
    );  
}  
  
@ParameterizedTest  
@MethodSource("exampleDateArrays")  
void correctDateOrder(LocalDate beginDate, LocalDate endDate) {  
    assertDoesNotThrow(() -> new MonthlyArray<>(beginDate, endDate));  
}
```

Parametryzowany test indeksu

```
private static Stream<Arguments> monthYearIndex() {  
    return Stream.of(  
        arguments(2012, 12, 35),  
        arguments(2022, 3, 146),  
        arguments(2020, 5, 124)  
    );  
}  
  
@ParameterizedTest  
@MethodSource("monthYearIndex")  
void paramMultipleIndices(int year, int month, int expectedIndex) {  
    MonthlyArray<Object> monthlyArray = new MonthlyArray<>(  
        LocalDate.of(2010, 1, 1),  
        LocalDate.of(2024, 1, 1)  
    );  
    assertEquals(expectedIndex, monthlyArray.index(year, month));  
}
```

Programowe generowanie parametrów

```
private static Stream<Arguments> allMonthYearIndices() {  
    Stream.Builder<Arguments> builder = Stream.<Arguments>builder();  
    int index = 0;  
    for (int year = 2010; year < 2024; year++)  
        for (int month = 1; month <= 12; month++) {  
            builder.add(arguments(year, month, index++));  
        }  
    return builder.build();  
}
```

```
@ParameterizedTest  
@MethodSource("allMonthYearIndices")  
void paramAllIndices(int year, int month, int expectedIndex) {  
    MonthlyArray<Object> monthlyArray = new MonthlyArray<>(  
        LocalDate.of(2010, 1, 1),  
        LocalDate.of(2024, 1, 1)  
    );  
    assertEquals(expectedIndex, monthlyArray.index(year, month));  
}
```

Parametryzacja plikiem CSV

```
@ParameterizedTest
@CsvFileSource(
    files = "months.csv",
    useHeadersInDisplayName = true
) void csvIndex(
    String date, int month, int year, int expectedIndex
) {
    MonthlyArray<Object> monthlyArray = new MonthlyArray<>(
        LocalDate.of(2010, 1, 1),
        LocalDate.of(2024, 6, 1)
    );
    assertEquals(
        expectedIndex,
        monthlyArray.index(year, month)
    );
}
```

	A	B	C	D
1	DATA	=MIESIĄC(A1)	=ROK(A1)	INDEX
2	1.01.2010	1	2010	0
3	1.02.2010	2	2010	1
4	1.03.2010	3	2010	2
5	1.04.2010	4	2010	3
6	1.05.2010	5	2010	4
7	1.06.2010	6	2010	5
8	1.07.2010	7	2010	6
9	1.08.2010	8	2010	7
10	1.09.2010	9	2010	8
11	1.10.2010	10	2010	9
12	1.11.2010	11	2010	10
13	1.12.2010	12	2010	11
14	1.01.2011	1	2011	12
15	1.02.2011	2	2011	13
16	1.03.2011	3	2011	14
17	1.04.2011	4	2011	15
18	1.05.2011	5	2011	16
19	1.06.2011	6	2011	17
20	1.07.2011	7	2011	18
21	1.08.2011	8	2011	19
22	1.09.2011	9	2011	20
23	1.10.2011	10	2011	21
24	1.11.2011	11	2011	22
25	1.12.2011	12	2011	23
26	1.01.2012	1	2012	24
27	1.02.2012	2	2012	25
28	1.03.2012	3	2012	26
29	1.04.2012	4	2012	27
30	1.05.2012	5	2012	28

Akcesor i mutator do tablicy

```
public class MonthlyArray<T> {  
    T data[];  
  
    public void set(int year, int month, T item) {  
        data[index(year, month)] = item;  
    }  
  
    public T get(int year, int month) {  
        return data[index(year, month)];  
    }  
}
```

Klasa z polem prywatnym bez akcesora

```
public class EmployeeAccount extends MonthlyArray<Double> {  
    private double tax;  
  
    public EmployeeAccount(LocalDate begin, LocalDate end) {  
        super(begin, end);  
    }  
  
    public void setTax(double tax) {  
        this.tax = tax;  
    }  
  
    @Override  
    public Double get(int year, int month) {  
        return super.get(year, month) * (1.0 - tax);  
    }  
}
```

~~Dostęp do pola prywatnego~~

```
public class EmployeeAccount extends MonthlyArray<Double> {  
    private double tax;  
}  
  
public class EmployeeAccountTest {  
    @Test  
    void testSetTax() {  
        EmployeeAccount account =  
            new EmployeeAccount(LocalDate.of(2023, 1, 1), LocalDate.of(2024, 1, 1));  
        account.setTax(0.2);  
        account.set(2023, 1, 1.0);  
        assertEquals(0.2, account.tax);  
    }  
}
```

Przetestowanie towarzyszących metod

```
public class EmployeeAccount extends MonthlyArray<Double> {  
    private double tax;  
    @Override  
    public Double get(int year, int month) {  
        return super.get(year, month) * (1.0 - tax);  
    }  
}  
  
public class EmployeeAccountTest {  
    @Test  
    void testSetTax() {  
        EmployeeAccount account =  
            new EmployeeAccount(LocalDate.of(2023, 1, 1), LocalDate.of(2024, 1, 1));  
        account.setTax(0.2);  
        account.set(2023, 1, 1.0);  
        Double result = account.get(2023, 1);  
        assertEquals(0.8, result);  
    }  
}
```

Dostęp pakietowy

```
package pl.umcs.arrays;  
public class EmployeeAccount extends MonthlyArray<Double> {  
    double tax;  
}
```

```
package pl.umcs.arrays;  
public class EmployeeAccountTest {  
    @Test  
    void testSetTax() {  
        EmployeeAccount account =  
            new EmployeeAccount(LocalDate.of(2023, 1, 1), LocalDate.of(2024, 1, 1));  
        account.setTax(0.2);  
        assertEquals(0.2, account.tax);  
    }  
}
```

Dostęp chroniony i klasa dziedzicząca

```
public class EmployeeAccount extends MonthlyArray<Double> {  
    protected double tax;  
}  
  
public class EmployeeAccountWithExposedTax extends EmployeeAccount {  
    public EmployeeAccountWithExposedTax(LocalDate begin, LocalDate end) {  
        super(begin, end);  
    }  
    double getTax() { return tax; }  
}  
  
public class EmployeeAccountTest {  
    @Test  
    void testSetTax() {  
        EmployeeAccountWithExposedTax account =  
            new EmployeeAccountWithExposedTax(LocalDate.of(2023, 1, 1), LocalDate.of(2024, 1,  
1));  
        account.setTax(0.2);  
        assertEquals(0.2, account.getTax());  
    }  
}
```

Tymczasowe upublicznienie pola

```
public class EmployeeAccount extends MonthlyArray<Double> {  
    private double tax;  
}  
  
public class EmployeeAccountTest {  
    @Test  
    void testSetTax() throws NoSuchFieldException, IllegalAccessException {  
        EmployeeAccount account =  
            new EmployeeAccount(LocalDate.of(2023, 1, 1), LocalDate.of(2024, 1, 1));  
        account.setTax(0.2);  
        Field taxField = EmployeeAccount.class.getDeclaredField("tax");  
        taxField.setAccessible(true);  
        double tax = taxField.getDouble(account);  
        assertEquals(0.2, tax);  
    }  
}
```

Metody niemutujące

```
public class EmployeeAccount extends MonthlyArray<Double>{

    public double sum() {
        double result = 0;
        for(int i=0; i<size(); ++i)
            if(super.get(i) != null)
                result += get(i);
        return result;
    }

    public double bestIncome() {
        double result = 0;
        for(int i=0; i<size(); ++i)
            if(super.get(i) != null && get(i) > result)
                result = get(i);
        return result;
    }
}
```


Zagnieżdżone testy

```
public class EmployeeAccountTest {  
    @Nested  
    class NonMutatingTest {  
        @Test  
        void testSum() {  
            EmployeeAccount account =  
                new EmployeeAccount(  
                    LocalDate.of(2023, 1, 1),  
                    LocalDate.of(2024, 1, 1)  
                );  
            account.setTax(0.2);  
            account.set(2023, 1, 1000.0);  
            account.set(2023, 2, 2000.0);  
            account.set(2023, 10, 1500.0);  
            double sum = account.sum();  
            assertEquals(3600, sum);  
        }  
    }  
}
```

```
@Test  
void testBestIncome() {  
    EmployeeAccount account =  
        new EmployeeAccount(  
            LocalDate.of(2023, 1, 1),  
            LocalDate.of(2024, 1, 1)  
        );  
    account.setTax(0.2);  
    account.set(2023, 1, 1000.0);  
    account.set(2023, 2, 2000.0);  
    account.set(2023, 10, 1500.0);  
    double result = account.bestIncome();  
    assertEquals(1600, result);  
}
```

Część wspólna wszystkich testów

```
public class EmployeeAccountTest {  
    @Nested  
    class NonMutatingTest {  
        static EmployeeAccount account;  
  
        @BeforeAll  
        static void prepareAccount() {  
            account = new EmployeeAccount(  
                LocalDate.of(2023, 1, 1),  
                LocalDate.of(2024, 1, 1)  
            );  
            account.setTax(0.2);  
            account.set(2023, 1, 1000.0);  
            account.set(2023, 2, 2000.0);  
            account.set(2023, 10, 1500.0);  
        }  
    }  
}
```

```
@Test  
void testSum() {  
    double sum = account.sum();  
    assertEquals(3600, sum);  
}  
  
@Test  
void testBestIncome() {  
    double result = account.bestIncome();  
    assertEquals(1600, result);  
}  
  
@AfterAll  
static void cleanup() {  
    /* ... */  
}
```

Metody mutujące

```
public class EmployeeAccount extends MonthlyArray<Double>{

    public void setMinimumWage(Double minimumWage) {
        for (int i = 0; i < size(); ++i) {
            if (super.get(i) == null || super.get(i) < minimumWage)
                super.set(i, minimumWage);
        }
    }

    public void addBonus(int year, int month, double bonus) {
        super.set(year, month, super.get(year, month) * bonus);
    }

}
```

Regeneracja danych przed każdym testem

```
public class EmployeeAccountTest {  
    @Nested  
    class NonMutatingTest {  
        EmployeeAccount account;  
  
        @BeforeEach  
        void prepareAccount() {  
            account = new EmployeeAccount(  
                LocalDate.of(2023, 1, 1),  
                LocalDate.of(2024, 1, 1)  
            );  
            account.setTax(0.2);  
            account.set(2023, 1, 1000.0);  
            account.set(2023, 2, 2000.0);  
            account.set(2023, 10, 1500.0);  
        }  
    }  
}
```

```
@Test  
void testMinimumWage() {  
    account.setMinimumWage(500.0);  
    assertEquals(800, account.get(2023, 1));  
    assertEquals(400, account.get(2023, 3));  
}  
  
@Test  
void testBonus() {  
    account.addBonus(2023, 1, 2.0);  
    assertEquals(1600, account.get(2023, 1));  
}  
  
@AfterEach  
static void cleanup() {  
    /* ... */  
}
```